

SAPIENZA UNIVERSITÀ DI ROMA

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Dipartimento di Ingegneria Informatica, Automatica e Gestionale "Antonio Ruberti"

Corso di Laurea Magistrale in Ingegneria Informatica

Tesi di Laurea Magistrale

Intelligent Packets

Inferenza neurale nel data plane eBPF/XDP:
un design space a tre pipeline e l'evoluzione della soluzione
hardcoded

Relatore

Prof. Marco Polverini

Candidato

Ettore Cantile

Matricola 2026562

Anno Accademico 2025/2026

Sommario

Questa tesi racconta il lavoro svolto sul progetto IPA (Intelligent Packets), un'infrastruttura che porta l'inferenza di una piccola rete neurale direttamente nel data plane del kernel Linux, usando eBPF e XDP. L'idea di fondo è che un router possa decidere l'inoltro di un pacchetto eseguendo un modello di machine learning per-hop, senza mai uscire dal kernel e senza interrogare un controller centrale.

Il lavoro si divide in due parti. La prima parte descrive un design space a tre pipeline (hardcoded, template, modular) che rappresentano tre modi diversi di implementare la stessa inferenza neurale nel datapath eBPF, con un preciso trade-off tra prestazioni di picco e flessibilità di aggiornamento a runtime. La seconda parte, più estesa, racconta come la pipeline hardcoded – quella che vince nettamente sul piano prestazionale – sia stata fatta evolvere per superare i suoi due limiti principali: un input vector cablato su un solo scenario di rete e un costo di ricompilazione molto alto ad ogni aggiornamento del modello. Vengono inoltre documentati gli esperimenti condotti per confrontare architetture più larghe contro architetture più profonde a parità di numero di pesi, e il lavoro di validazione della metodologia di misura, che si è rivelato tutt'altro che banale.

Indice

Sommario	1
1 Introduzione a eBPF e XDP	3
1.1 Una macchina virtuale dentro il kernel	3
1.2 I punti di aggancio e i mattoni principali	3
2 Il paradigma Intelligent Packets: la prima dimostrazione	6
2.1 L'idea di fondo	6
2.2 Lo switch neurale	6
2.3 Il problema dell'allineamento e i quattro metodi	7
2.3.1 Metodo 1: Post-Training Quantization (PTQ)	8
2.3.2 Metodo 2: Quantization-Aware Training (QAT)	8
2.3.3 Metodo 3: OpenFlow-like	8
2.3.4 Metodo 4: IPA Demo	9
2.4 Alcuni dettagli implementativi che ci hanno fatto perdere tempo	9
2.5 Cosa abbiamo imparato	10
3 Ambiente sperimentale	11
3.1 Kathara e l'emulazione di rete	11
3.2 La topologia Germany50	11
3.3 Perché non tutto il traffico di rete arriva dove ci si aspetterebbe	12
4 Il design space delle pipeline neurali	13
4.1 Architettura comune alle tre pipeline	13
4.2 Pipeline 1: hardcoded	14
4.3 Pipeline 2: template	15
4.4 Pipeline 3: modular	15
4.5 Dall'argmax all'azione: una mappa comune	16
4.6 Risultati sperimentali comparativi	16
4.7 Come sono cambiati i numeri nel tempo	18
4.8 Il costo della flessibilità	19

5	L'evoluzione della pipeline hardcoded	20
5.1	I due limiti di partenza	20
5.2	Un input vector generico	21
5.2.1	Coerenza tra checkpoint e topologia	22
5.2.2	Un esempio concreto	22
5.2.3	Ottimizzazione dell'hot path: da N letture a una sola	23
5.3	AOT-literal: separare la compilazione dal deploy	23
5.4	Profondità variabile	25
5.5	Larghezza contro profondità: uno studio sperimentale	25
5.5.1	Metodologia	25
5.5.2	Il problema del rumore di misura	26
5.5.3	Risultati	27
5.6	Isolare il costo puro della tail call	28
5.7	AOT-literal in sostituzione di BCC per il deploy	28
5.8	Robustezza del control plane: la risoluzione dei MAC per classe	29
6	Metodologia di validazione e onestà delle misure	31
6.1	BPF_PROG_TEST_RUN come banco di prova indipendente dalla rete	31
6.2	Perché una sola architettura non basta	32
6.3	Cosa dice la letteratura sulla valutazione di un datapath eBPF/XDP	32
6.4	Alcuni tentativi abbandonati	33
7	Conclusioni e sviluppi futuri	34

Capitolo 1

Introduzione a eBPF e XDP

1.1 Una macchina virtuale dentro il kernel

Prima di entrare nel merito del progetto è necessario capire su cosa è costruito tutto il resto di questo lavoro: eBPF (extended Berkeley Packet Filter). eBPF è un meccanismo del kernel Linux che permette di caricare ed eseguire piccoli programmi direttamente dentro il kernel stesso, agganciandoli a eventi di sistema o di rete, senza dover scrivere un modulo kernel e senza dover ricompilare o riavviare nulla.

Quello che rende interessante eBPF non è solo la possibilità di eseguire codice arbitrario nel kernel – cosa che, detta così, suonerebbe pericolosa – ma il fatto che questo avvenga dentro una sandbox controllata. Prima che un programma eBPF venga caricato, un componente chiamato *verifier* lo analizza staticamente e rifiuta qualunque cosa possa essere pericolosa: loop non limitati, accessi fuori dai limiti di un buffer, puntatori non validati. Solo se il verifier approva, il bytecode eBPF viene compilato JIT in istruzioni macchina native e agganciato al punto di hook richiesto, eseguendo poi a velocità nativa.

Il ciclo di vita di un programma eBPF si può riassumere in cinque passi: si scrive il programma in un sottoinsieme ristretto del linguaggio C, lo si compila in bytecode eBPF, il verifier lo controlla, il JIT lo traduce in codice macchina e infine lo si aggancia a un evento – una syscall, un pacchetto di rete, un tracepoint – che ne innesci l'esecuzione ad ogni occorrenza.

1.2 I punti di aggancio e i mattoni principali

eBPF può agganciarsi in diversi punti del kernel. Un kprobe intercetta l'ingresso o l'uscita da una funzione del kernel o da una syscall; è tipicamente usato per tracing e debug. Le maps sono strutture dati che vivono nel kernel ma sono leggibili e scrivibili sia da altri programmi eBPF sia da user space: sono il modo standard per mantenere

stato tra un'esecuzione e l'altra, o per far comunicare kernel e user space. XDP (eXpress Data Path) è il punto di aggancio più vicino alla scheda di rete: il programma eBPF viene eseguito nel driver stesso, prima ancora che il kernel costruisca le sue strutture dati per il pacchetto, ed è quindi il punto più veloce in assoluto per ispezionare, scartare o reindirizzare un pacchetto. Un programma XDP può restituire quattro verdetti principali: `XDP_PASS` (il pacchetto prosegue verso lo stack di rete), `XDP_DROP` (il pacchetto viene distrutto), `XDP_TX` (il pacchetto torna indietro sulla stessa interfaccia) e `XDP_REDIRECT` (il pacchetto viene deviato su un'altra interfaccia).

Un altro meccanismo fondamentale per tutto il progetto è la *tail call*: un programma eBPF può cedere il controllo a un altro programma eBPF a runtime, tramite una struttura chiamata `BPF_PROG_ARRAY` e la primitiva `bpf_tail_call()`. Se il salto riesce, l'esecuzione non torna mai indietro al programma chiamante – è più simile a una `exec` che a una chiamata di funzione. Questo meccanismo è quello che permette di costruire pipeline modulari nel kernel, spezzando la logica in più programmi più piccoli invece di scrivere un unico programma monolitico, cosa che tra l'altro il verifier a volte impone: durante questo lavoro abbiamo incontrato più di un caso in cui un singolo blocco di codice troppo grande semplicemente non passava la verifica per limiti di complessità, e l'unica soluzione è stata spezzarlo in più tail call.

Un ultimo dettaglio che torna continuamente nel seguito è quella che in una delle presentazioni preparate durante il progetto abbiamo chiamato la “regola d'oro” del parsing dei pacchetti: ogni volta che si accede a un campo di un header, bisogna prima verificare esplicitamente che il puntatore non superi `data_end`, altrimenti il verifier rifiuta il programma. Questo perché XDP lavora su un buffer di memoria di dimensione variabile e non ha alcuna garanzia a priori su quanti byte siano effettivamente disponibili.

Listing 1.1: La regola d'oro: controllo dei limiti prima di ogni accesso

```
1 void *data      = (void *) (long) ctx->data;
2 void *data_end  = (void *) (long) ctx->data_end;
3 struct ethhdr *eth = data;
4 if ((void *) (eth + 1) > data_end)
5     return XDP_PASS;
6 if (eth->h_proto == bpf_htons(ETH_P_IP)) {
7     struct iphdr *ip = (void *) (eth + 1);
8     if ((void *) (ip + 1) > data_end)
9         return XDP_PASS;
10 }
```

Su questi mattoni – maps, XDP, tail call, parsing sicuro dei pacchetti – è costruito tutto quello che segue: un protocollo applicativo personalizzato incapsulato dentro UDP (un header “IPA” che viaggia dopo l'header UDP, su una porta convenzionale),

e sopra questo protocollo, un intero sistema di inferenza neurale eseguito interamente nel kernel.

Capitolo 2

Il paradigma Intelligent Packets: la prima dimostrazione

2.1 L'idea di fondo

Il progetto IPA nasce da un'idea abbastanza radicale rispetto al modo classico di pensare a un router programmabile: invece di avere un controller centrale che calcola le regole di forwarding e le installa nei nodi, è il pacchetto stesso a portare con sé l'informazione necessaria per essere instradato. Un modello di machine learning compatto – nel nostro caso un MLP con pesi quantizzati a 8 bit – viene incorporato nell'header del pacchetto o registrato nel nodo, e ad ogni salto il router esegue una mini-inferenza locale per decidere su quale interfaccia inoltrare il pacchetto.

L'idea centrale, per come l'abbiamo formulata nelle prime presentazioni del lavoro, è che il pacchetto non trasporti solo dati ma anche l'informazione per scegliere il proprio percorso, e che il router esegua questa decisione in autonomia, senza mai dover interrogare un controller esterno. Missioni di rete diverse – recupero da un guasto, rispetto di una deadline, evitare la congestione – diventano semplicemente modelli diversi caricati nel pacchetto o nel nodo.

2.2 Lo switch neurale

La prima implementazione concreta di questa idea è uno switch XDP, che abbiamo chiamato `switch_core`, organizzato attorno a due mappe eBPF. La prima, `model_cache`, associa un identificativo di modello (`model_id`) ai pesi della rete neurale corrispondente. La seconda, `fwd_table`, associa il risultato dell'inferenza a un'azione di forwarding vera e propria: l'interfaccia di uscita e i MAC address da riscrivere prima del redirect.

Il flusso è semplice: arriva un pacchetto con un header IPA custom che dichiara un `model_id`; lo switch cerca quel `model_id` in `model_cache` – se non lo trova, o se la

entry non è valida, scarta il pacchetto (CACHE MISS $\rightarrow$ XDP_DROP); se lo trova, esegue l'inferenza, che nella prima versione è un semplice prodotto scalare tra un piccolo input vector (composto da `ingress_ifindex`, TTL del pacchetto e un paio di costanti) e i pesi della rete, srotolato a mano perché il verifier non accetta loop il cui numero di iterazioni non sia staticamente noto. Il risultato del prodotto scalare diventa la chiave con cui si interroga `fwd_table`: se la chiave esiste, lo switch riscrive gli indirizzi MAC e fa `bpf_redirect()` sull'interfaccia indicata; altrimenti scarta il pacchetto.

Listing 2.1: Il cuore dello switch neurale: inferenza srotolata e redirect

```
1 long long in[4];
2 in[0] = ctx->ingress_ifindex;
3 in[1] = ip->ttl;
4 in[2] = 1; in[3] = 1;
5 long long out = 0;
6 out += in[0] * (__s8)w[0];
7 out += in[1] * (__s8)w[1];
8 out += in[2] * (__s8)w[2];
9 out += in[3] * (__s8)w[3];
10
11 struct fwd_action *act = fwd_table.lookup(&out);
12 if (act != NULL) {
13     __builtin_memcpy(eth->h_source, act->src_mac, 6);
14     __builtin_memcpy(eth->h_dest, act->dst_mac, 6);
15     return bpf_redirect(act->ifindex, 0);
16 }
17 return XDP_DROP;
```

2.3 Il problema dell'allineamento e i quattro metodi

Il punto delicato di questa architettura è che ci sono due posti distinti in cui viene calcolata (o preconfigurata) la stessa informazione: il control plane (CP), tipicamente uno script Python in user space, che popola `fwd_table` in anticipo con le chiavi che si aspetta il modello produca, e il kernel, che ricalcola la stessa chiave a runtime con aritmetica intera. Se questi due calcoli divergono anche di un solo bit, la chiave calcolata dal kernel non trova corrispondenza in `fwd_table` e il pacchetto viene scartato o instradato in modo scorretto. Per esplorare quanto questo problema fosse serio e come risolverlo, sono stati implementati e confrontati quattro metodi diversi, tutti testati inviando lotti di 30 pacchetti e classificando ogni pacchetto come TRUE HIT (la chiave calcolata dal kernel coincide con quella attesa), FAKE HIT (in tabella c'è una chiave,

ma non è quella corretta per quel TTL) o MISS (la chiave non è affatto presente in tabella).

2.3.1 Metodo 1: Post-Training Quantization (PTQ)

Nel primo metodo il modello viene addestrato in virgola mobile (float32) e solo dopo l'addestramento i pesi vengono quantizzati a interi a 8 bit per l'uso nel kernel. Il control plane, però, popola `fwd_table` usando ancora i pesi float originali, mentre il kernel esegue l'aritmetica con i pesi int8 caricati in `model_cache`. Questa asimmetria è voluta: serve proprio a misurare quanto la quantizzazione a posteriori rompa l'allineamento tra i due calcoli. I risultati sono netti e per certi versi prevedibili: su 30 pacchetti di test, 0 TRUE HIT, 18 FAKE HIT (60%) e 12 MISS (40%). In pratica il PTQ, in questa architettura, non è utilizzabile: l'errore di quantizzazione a posteriori è troppo grande perché le chiavi calcolate in float e quelle ricalcolate in int8 coincidano quasi mai.

2.3.2 Metodo 2: Quantization-Aware Training (QAT)

Il secondo metodo cambia strategia: il modello viene addestrato simulando già durante il training l'effetto della quantizzazione, con la tecnica dello straight-through estimator. Ad ogni passo di training i pesi vengono scalati per un fattore fisso (128), arrotondati, portati nell'intervallo $[-128, 127]$ e poi riportati in float per continuare il training, ma tenendo conto dell'errore introdotto. Sia il control plane sia il kernel usano poi lo stesso `SCALE_FACTOR` fisso e la stessa divisione intera (in Python l'operatore `//`, per replicare esattamente la semantica della divisione intera in C). Il risultato è quasi un ribaltamento completo rispetto al PTQ: 30 TRUE HIT su 30 pacchetti, 0 FAKE HIT, 0 MISS. Con la stessa identica aritmetica intera su entrambi i lati, le chiavi coincidono sempre.

2.3.3 Metodo 3: OpenFlow-like

Il terzo metodo cambia il modo in cui `fwd_table` viene popolata. Invece di precaricare tutte le regole in anticipo, la tabella parte vuota: quando il kernel calcola una chiave che non trova, invia un evento di MISS al control plane, che installa la regola usando *direttamente* la chiave già calcolata dal kernel – non un ricalcolo lato user space, eliminando così ogni possibilità di mismatch per costruzione. È lo stesso principio che sta dietro a OpenFlow: le regole si installano su richiesta, reattivamente. Uno snapshot preso durante la fase di warm-up mostra 5 TRUE HIT (17%), 14 FAKE HIT (47%) e 11 MISS (37%): numeri che sembrano peggiori del QAT, ma che vanno letti insieme al fatto che questo è proprio il costo del primo apprendimento. Una volta che ogni TTL

distinto è stato incontrato almeno una volta, i MISS si azzerano e i TRUE HIT crescono stabilmente, senza che sia stato necessario cablare a priori nessun intervallo di TTL.

2.3.4 Metodo 4: IPA Demo

Il quarto metodo è quello concettualmente più vicino al paradigma originale di IPA: sia `model_cache` sia `fwd_table` partono completamente vuote all'avvio. Il router non conosce nessun modello finché non arriva il primo pacchetto per quel `model_id`: quel primo pacchetto porta nel proprio payload i pesi della rete (nel prototipo, 4 byte, uno per peso). Il kernel genera un evento di MODEL MISS, il control plane estrae i pesi dal payload, popola `model_cache` e installa la prima regola di forwarding, in un tempo dell'ordine di 2 millisecondi. Da quel momento in poi, per quel modello, ogni pacchetto viene gestito interamente dal kernel in meno di un millisecondo, senza che il control plane debba più intervenire. È il caso che rappresenta più fedelmente l'idea di partenza: il pacchetto porta con sé l'intelligenza, il router la apprende al volo e poi diventa autonomo.

Un dettaglio implementativo interessante di questo metodo riguarda proprio il verifier: leggere un numero variabile di byte dal payload con un loop non è accettato, perché il verifier non riesce a limitare staticamente il numero di iterazioni in relazione al controllo su `data_end`. La soluzione adottata è stata leggere un numero fisso di byte (4, nel prototipo) con un unico controllo di limite, evitando del tutto il loop.

2.4 Alcuni dettagli implementativi che ci hanno fatto perdere tempo

Prima di passare al design space vale la pena raccontare qualche dettaglio implementativo minore, di quelli che non finiscono mai in una presentazione ma che in pratica hanno richiesto più tempo di debug del previsto, perché tornano utili anche più avanti.

Il primo riguarda BCC e i tipi kernel. Quando il kernel invia un evento a user space attraverso una perf ring buffer (come i MISS EVENT dei metodi 3 e 4), BCC dovrebbe generare automaticamente, a partire dalla struct C definita nel programma eBPF, una classe Python equivalente per leggere l'evento. Nella pratica, però, BCC non è sempre in grado di farlo correttamente quando la struct usa tipi come `__u8` o `__u64`, che non riconosce in ogni contesto: la soluzione adottata è stata scrivere a mano, con `ctypes.Structure`, le classi Python corrispondenti (`MissEvent` e `ModelMissEvent`), invece di affidarsi alla generazione automatica.

Il secondo dettaglio, strettamente collegato al primo, riguarda il padding implicito che il compilatore C inserisce nelle struct. Una struct che, contando solo i campi dichiarati, dovrebbe occupare un certo numero di byte, in pratica ne occupa qualcuno

in più per via dell'allineamento imposto dal compilatore: nel nostro caso, due byte di padding tra alcuni campi e altri sette in coda, per un totale di 24 byte invece dei 15 che ci si aspetterebbe sommando solo i campi. Per replicare esattamente questo layout lato Python, le classi `ctypes.Structure` usate per decodificare gli eventi dichiarano `_pack_ = 1` e alcuni campi di padding espliciti (`_pad`), altrimenti la lettura degli eventi lato user space produce valori completamente sbagliati, letti dagli offset sbagliati.

Il terzo dettaglio, più concettuale, è l'errore di arrotondamento di ± 1 che compare quando si confronta una divisione in virgola mobile con la divisione intera del kernel: in Python, l'operatore `//` implementa esattamente la stessa semantica di troncamento della divisione intera in C, e usarlo al posto della normale divisione (`/`) nel control plane è quello che permette, nei metodi 2, 3 e 4, di ottenere esattamente la stessa chiave calcolata dal kernel; nel metodo 1 questa stessa divergenza è invece lasciata deliberatamente, proprio per misurarne l'effetto.

Infine, un vincolo del verifier che si ripresenterà più volte nel resto di questa tesi: il verifier non accetta aritmetica di puntatori il cui limite superiore non sia verificabile staticamente rispetto a `data_end`, il che esclude i normali cicli di lettura a lunghezza variabile. Ogni volta che serve leggere un numero di byte che in linea di principio potrebbe variare, la soluzione è la stessa vista per il metodo 4: un numero fisso di letture a offset costante, precedute da un unico controllo di limite.

2.5 Cosa abbiamo imparato

Mettendo a confronto i quattro metodi si vede chiaramente che il problema centrale non è la quantizzazione in sé, ma *dove* e *come* viene calcolata la stessa informazione: nel PTQ il control plane e il kernel calcolano cose diverse (float contro int8) e il disallineamento è quasi garantito; nel QAT calcolano la stessa cosa con la stessa aritmetica, e l'allineamento è perfetto; nei metodi 3 e 4 il problema si elimina proprio alla radice, facendo sì che sia sempre il kernel a produrre la chiave, e il control plane si limiti a installarla.

Questa distinzione – popolamento statico delle tabelle contro popolamento reattivo, aritmetica calcolata due volte contro aritmetica calcolata una volta sola e propagata – è la lezione che ha guidato la fase successiva del progetto, quella descritta nei capitoli seguenti: un design space che generalizza l'idea a un modello neurale multi-strato vero e proprio (non più un singolo prodotto scalare a 4 termini), esplorando sistematicamente il compromesso tra quanto il codice del datapath viene specializzato per un singolo modello e quanto invece resta generico e aggiornabile a runtime.

Capitolo 3

Ambiente sperimentale

Prima di entrare nel merito del design space a tre pipeline, che è il cuore sperimentale di questa tesi, vale la pena descrivere l’ambiente in cui tutti gli esperimenti sono stati condotti, perché diversi limiti e scelte descritti nei capitoli successivi si capiscono meglio conoscendo questo contesto.

3.1 Kathara e l’emulazione di rete

Kathara è uno strumento di emulazione di reti che permette di far girare una topologia complessa facendo corrispondere ogni nodo a un container Docker leggero, invece che a una macchina virtuale completa. La differenza non è di poco conto per questo progetto: un container condivide il kernel del sistema host, e questo è esattamente ciò che rende possibile eseguire programmi eBPF/XDP dentro ciascun nodo emulato, cosa che non sarebbe altrettanto naturale con macchine virtuali pesanti indipendenti. Ogni nodo Kathara usato in questo lavoro esegue, all’avvio, uno script di inizializzazione (`fix_bpf.sh`) che monta il filesystem `debugfs`, richiesto da alcuni strumenti diagnostici eBPF, abilita l’inoltro IP e avvia il demone di routing FRR con il protocollo OSPF.

3.2 La topologia Germany50

La topologia usata per tutti gli esperimenti è *Germany50*, importata dal repository SNDlib (una libreria di topologie di rete reali usata comunemente in letteratura per esperimenti di networking) tramite uno script di conversione (`importSNDLib.py`). Si tratta di una topologia di dorsale con 50 nodi, a cui si aggiungono due host virtuali di sorgente e destinazione per un totale di 52 nodi, con un grado massimo di un singolo nodo pari a 6 – un numero che non è casuale, perché è esattamente il numero di interfacce di uscita che il modello neurale usato in tutto questo lavoro deve poter

scegliere, e quindi il numero di classi in uscita dall'argmax finale (sei interfacce più la classe di scarto).

Nella configurazione usata per la prima dimostrazione del paradigma IPA (Capitolo 2), il traffico di test viaggia da un host sorgente collegato a Karlsruhe fino a un host di destinazione collegato a Flensburg, attraversando come router intermedio il nodo Frankfurt, su cui viene effettivamente caricato il programma XDP che esegue l'inferenza; il nodo Darmstadt è tipicamente usato come sorgente dei pacchetti di test generati con Scapy, verso la porta UDP 9999 su cui è in ascolto il protocollo IPA.

3.3 Perché non tutto il traffico di rete arriva dove ci si aspetterebbe

Un limite pratico di questo fabric, incontrato lavorando alla validazione delle pipeline (se ne parla più diffusamente nel Capitolo 6), merita di essere segnalato già qui: il traffico UDP sulla porta 9999 non viene sempre consegnato end-to-end come ci si aspetterebbe. In particolare, indirizzare un pacchetto verso l'indirizzo di loopback di un nodo che si trova a più di un salto di distanza non funziona – i pacchetti vengono inviati correttamente dal mittente, ma non arrivano mai a destinazione, senza che il mittente riceva alcun errore, dato che UDP è per natura un protocollo senza conferma di consegna. Indirizzare invece un pacchetto direttamente all'indirizzo IP dell'interfaccia di un vicino connesso a un solo salto di distanza funziona correttamente. Questo comportamento, scoperto empiricamente confrontando l'output di un generatore di traffico con una cattura `tcpdump` sul nodo ricevente, ha condizionato non poco le scelte di validazione descritte più avanti in questa tesi, ed è anche uno dei motivi per cui alcuni tentativi di test più ambiziosi, raccontati nel Capitolo 6, non sono arrivati a un risultato pienamente affidabile.

Capitolo 4

Il design space delle pipeline neurali

Il cuore sperimentale di questa tesi è un design space a tre punti: tre modi diversi di eseguire *lo stesso* modello neurale – un MLP con architettura 65 input, due hidden layer da 4 neuroni ciascuno, 7 output – dentro il datapath eBPF/XDP. I tre punti si chiamano, nel codice e in questa tesi, P1 hardcoded, P2 template e P3 modular, e si differenziano per quanto il codice generato sia specializzato sul singolo modello rispetto a quanto sia generico e parametrizzabile a runtime. Come si vedrà, più si specializza il codice più aumenta la velocità di picco ma diminuisce la flessibilità; viceversa, aumentando la flessibilità crescono il numero di tail call, di letture da mappa e di istruzioni eBPF eseguite per pacchetto, e il throughput massimo cala di conseguenza.

4.1 Architettura comune alle tre pipeline

Le tre pipeline condividono lo stesso protocollo applicativo e lo stesso modello. Il pacchetto IPA viaggia dentro UDP sulla porta 9999 e porta un header custom con `model_id`, `scale_factor` e un descrittore delle feature usate (su cui torneremo diffusamente nel Capitolo 5). Il programma XDP intercetta il pacchetto in ingresso, prima ancora che raggiunga lo stack di rete Linux, e in tutti e tre i casi il modello di riferimento è lo stesso MLP 65→4→4→7, quantizzato post-training a interi a 8 bit, con aritmetica interamente intera nel kernel – nessun float compare mai nel codice eseguito in kernel space. L'output finale è un argmax su 7 classi: sei rappresentano le interfacce di uscita fisiche, la settima è la classe di scarto (**DROP**).

L'input vector di 65 componenti è costruito da quattro tipi di feature concatenate: le prime 6 componenti (`link_state`) rappresentano lo stato up/down delle sei interfacce di uscita del nodo; le successive 6 (`iface`, codificata one-hot) indicano da quale porta è arrivato il pacchetto; la tredicesima componente è il TTL del pacchetto, letto come scalare; le ultime 52 componenti (`node`, anch'essa one-hot) identificano il nodo corrente all'interno della topologia. Il vettore è quindi molto sparso: a runtime solo una manciata di posizioni sono effettivamente diverse da zero.

Un aspetto su cui abbiamo lavorato esplicitamente è rendere vera la prima feature, `link_state`. In una prima versione questa feature era sempre a 1 (tutti i link sempre su), il che significa che il modello non vedeva mai un guasto reale e quindi non poteva mai imparare a reagire ad esso. Abbiamo introdotto un thread in user space, `link_state_monitor.py`, che interroga periodicamente lo stato del carrier di ogni interfaccia (leggendo `/sys/class/net/ethX/carrier`) e scrive il valore corrispondente in una mappa BPF condivisa, `link_state[0..5]`. I programmi eBPF leggono questo valore ad ogni pacchetto, senza mai dover essere ricaricati: se un'interfaccia cade, la sua feature passa a 0 al polling successivo, e questo può far sì che l'argmax scelga un'uscita diversa – è il meccanismo che nella documentazione del progetto chiamiamo *fast-reroute*. All'avvio la mappa viene inizializzata a tutti i link su, come stato di base sano.

4.2 Pipeline 1: hardcoded

Nella pipeline hardcoded i pesi della rete neurale sono incorporati come letterali direttamente nel sorgente C che viene generato e compilato per quel modello specifico. Un dispatcher molto leggero legge il `model_id` dal pacchetto ed esegue un'unica tail call verso il programma `model_<id>` corrispondente; da lì in poi l'intera inferenza – costruzione dell'input vector, i due hidden layer, l'output layer, l'argmax – è completamente srotolata, senza un solo ciclo e senza una sola lettura da mappa per i pesi. Le uniche letture da mappa che sopravvivono sono quelle di `link_state` (che è una feature di input, non un peso) e, a valle dell'argmax, quella su `mac_table` per risolvere l'azione di forwarding.

Un dettaglio tecnico non banale riguarda le due feature codificate one-hot (`iface` e `node`). L'idea più naturale – un array di pesi statico indicizzato da una variabile calcolata a runtime – non è accettata dal verifier: un `static const` indicizzato da una variabile non risolvibile staticamente genera una relocation che, in fase di caricamento, collassa a zero, con il risultato che il modello si comporterebbe come se quella feature non contasse mai nulla, senza che il verifier segnali nulla di anomalo a compile time. La soluzione adottata è un unico blocco `switch` per ciascuna feature one-hot (uno per `iface`, uno per `node`), che assegna in un colpo solo il contributo di quella feature a tutti i neuroni del primo hidden layer, invece di usare uno switch per ogni singolo neurone: quest'ultima soluzione, più naturale da scrivere, genera una esplosione combinatoria di rami che il verifier arriva a rifiutare per limite di complessità – abbiamo effettivamente incontrato un caso concreto in cui un blocco denso unico per la pipeline modular compilava a quasi 20000 istruzioni, ben oltre il limite di 4096 imposto dal kernel usato in laboratorio, ed è stato necessario spezzarlo (se ne parla nella sezione dedicata alla pipeline modular).

Il costo di aggiornare un modello in questa pipeline è, semplicemente, il costo di rigenerare il sorgente C, ricompilarlo e ricaricare l'intero programma XDP: non esiste alcuna via incrementale, per costruzione, dato che i pesi sono letterali nel codice sorgente. È la specializzazione massima e, come vedremo nei risultati sperimentali, anche le prestazioni massime.

4.3 Pipeline 2: template

La pipeline template sposta i pesi fuori dal codice sorgente e dentro una mappa BPF (`arch_weights`, un `BPF_ARRAY`), lasciando invece fisso il *codice* che implementa una intera famiglia di architetture: tutti i modelli con la stessa struttura generale – due hidden layer, input e output di dimensione fissata dal protocollo – condividono lo stesso programma compilato, `arch_generic_2layer`. Le larghezze dei due hidden layer, `n_h1` e `n_h2`, sono lette a runtime da una mappa di registro (`arch_registry`), fino a un tetto massimo fissato in fase di compilazione: cambiare la larghezza di un modello, o registrarne uno completamente diverso ma con la stessa profondità, non richiede alcuna ricompilazione, solo una scrittura nelle mappe dei pesi e del registro.

Il prodotto scalare del primo hidden layer, invece di sommare termini letterali come in P1, itera su un ciclo srotolato che ad ogni iterazione fa una lettura dalla mappa `arch_weights` a un offset calcolato dinamicamente da `n_h1/n_h2`. Il fatto che l'accesso sia su una mappa BPF regolarmente registrata, e non su un array statico nel codice, è proprio ciò che permette al verifier di accettare un indice calcolato a runtime: l'aritmetica dell'indice è sicura perché la mappa è un oggetto kernel con dimensioni note, non una relocation a compile time.

Abbiamo verificato concretamente questa flessibilità registrando, nello stesso oggetto BPF compilato, due modelli con forma diversa: il modello reale 65-4-4-7 insieme a un modello sintetico 65-6-5-7 (stessa profondità, larghezza diversa), ottenendo corrispondenza corretta con il riferimento Python per entrambi.

4.4 Pipeline 3: modular

La pipeline modular porta il principio della pipeline template ancora oltre, rendendo dinamica non solo la larghezza ma anche la *profondità* del modello. L'inferenza è scomposta in una catena di tail call attraverso due soli programmi generici: `layer_first`, sempre eseguito al primo hop, che fa una lettura sparsa del vettore di 65 feature esattamente come in P1/P2, e `layer_hidden`, eseguito per ogni hop successivo, che fa invece un prodotto denso $n_{in} \rightarrow n_{out}$ tra le dimensioni lette a runtime da un registro (`layer_registry/layer_shapes`). Le attivazioni intermedie – l'output di un layer che

serve da input al layer successivo – vengono scritte in una mappa di scratch per-CPU (`PERCPU_ARRAY`), in modo che CPU diverse non si contendano mai lo stesso stato.

Un punto concettualmente importante è che quale sia l'*ultimo* layer non è deciso da quale slot della tail-call array è occupato, ma dai dati stessi: ogni hop controlla se il proprio indice di layer più uno coincide con il numero totale di layer del modello corrente, letto dal registro. Questo permette a modelli con profondità diversa di condividere esattamente la stessa catena di tail call senza nessun conflitto. Anche qui abbiamo validato concretamente il caso limite: due modelli registrati nello stesso oggetto compilato, quello reale a 3 layer (65-4-4-7) e uno sintetico a 4 layer (65-5-6-4-7, sia profondità sia larghezza diverse), entrambi corretti.

È in questa pipeline che, come accennato, abbiamo incontrato concretamente il limite di complessità del verifier: un primo tentativo di scrivere un unico blocco denso per l'intera inferenza compilava a circa 19983 istruzioni, contro un limite di 4096 imposto dal kernel di laboratorio. La soluzione, oltre a confermare la scelta architetturale della tail call, è stata proprio separare `layer_first` (che fa la lettura sparsa iniziale, relativamente leggera) da `layer_hidden` (che fa il prodotto denso, ripetuto quante volte serve via tail call), tenendo ciascun pezzo sotto il limite.

Il prezzo di questa flessibilità massima è, ovviamente, un numero di tail call per pacchetto che dipende dalla profondità del modello – tre, per il modello di riferimento a tre layer – contro l'unica tail call di P1 e P2.

4.5 Dall'argmax all'azione: una mappa comune

Una volta calcolato l'argmax, le tre pipeline condividono lo stesso schema per tradurlo in un'azione di rete concreta: la classe scelta (0..5 per le interfacce, 6 per `DROP`) viene usata come chiave in una mappa `mac_table`, che restituisce l'indice dell'interfaccia di uscita insieme ai MAC address sorgente e destinazione da scrivere nell'header Ethernet prima del redirect. In P1 questa risoluzione avviene tramite uno switch cablato (zero letture da mappa aggiuntive); in P2 e P3 tramite un'unica lettura sulla mappa `mac_table`. Non viene fatta nessuna validazione ulteriore sull'output del modello: ci si fida della decisione della rete neurale, coerentemente con lo spirito del paradigma IPA descritto nel capitolo precedente.

4.6 Risultati sperimentali comparativi

Tutte e tre le pipeline sono state misurate con la stessa metodologia: i programmi XDP reali vengono caricati nel kernel e interrogati con `BPF_PROG_TEST_RUN`, la primitiva che permette di eseguire un programma eBPF su un pacchetto sintetico direttamente dal kernel, senza dover attraversare una vera scheda di rete. Questo è stato necessario

perché il fabric del laboratorio Kathara, descritto meglio nel Capitolo 6, non consegna traffico UDP sulla porta 9999 end-to-end in ogni condizione, e quindi la verifica di correttezza va fatta a questo livello. Accanto alle tre pipeline è stata misurata anche una quarta configurazione, chiamata *baseline*: un programma che fa lo stesso parsing del dispatcher e lo stesso redirect finale, ma senza nessuna tail call e senza nessuna inferenza – serve da pavimento di riferimento per capire quanto della latenza osservata sia dovuto davvero al modello neurale e quanto al solo framework XDP.

La Tabella 4.1 riporta i numeri più recenti e più affidabili raccolti durante il lavoro di validazione descritto nel Capitolo 6, ottenuti con una metodologia a più campioni indipendenti (se ne spiega il perché più avanti).

Tabella 4.1: Confronto sperimentale tra le quattro configurazioni (kernel reale, BPF_PROG_TEST_RUN, minimo su 7 trial indipendenti)

Metrica	baseline	P1 hardcoded	P2 template	P3 modular
Istruzioni eBPF (xlated)	113	980	16 988	15 767
Codice jited (byte)	542	4 684	84 587	76 376
Tail call / pacchetto	0	1	1	3
Lecture mappa / pacchetto (reali)	0	4.0	147.0	160.0
Memoria mappe (byte)	280	308	8 052	16 884
Latenza minima (ns/pacchetto)	21.0	35.0	262.0	442.0
Throughput (Mpps, da latenza minima)	47.619	28.571	3.817	2.262
CPU (%)	34	30	57	51

La lettura è coerente con la tassonomia proposta: il costo – in istruzioni, codice jited, tail call, letture di mappa e memoria – cresce in modo monotono passando da baseline a P1 a P2 a P3, e le prestazioni calano nello stesso ordine. Il baseline, che rappresenta il solo pavimento di parsing e redirect senza nessuna inferenza, misura 21 nanosecondi per pacchetto e 47.6 milioni di pacchetti al secondo; la pipeline hardcoded aggiunge circa 14 nanosecondi per la tail call, il doppio parsing che essa comporta e l'intera rete neurale, arrivando a 35 nanosecondi – circa 1.7 volte più lenta del pavimento assoluto, non ordini di grandezza. Questo numero è importante perché risponde direttamente a una domanda posta esplicitamente dal relatore, cioè se il throughput molto alto misurato per la pipeline hardcoded fosse in qualche modo sospetto: il confronto con il baseline mostra che quel throughput alto non è altro che il pavimento imposto dal framework XDP stesso – una rete int8 65-4-4-7 completamente srotolata costa pochissimo in confronto.

Un'osservazione interessante riguarda P2 e P3: pur avendo un conteggio statico di istruzioni molto simile (circa 17000 e 15800 rispettivamente), le loro prestazioni differiscono in modo netto, e la differenza si spiega quasi interamente con il numero di tail call (tre contro una) e con le letture di mappa aggiuntive (163 contro 150 nella versione storica), non con il volume di codice in sé.

È stato inoltre misurato, con lo script `bench_model_add.py`, il costo reale – non stimato, ma misurato con chiamate vere a `BPF_PROG_LOAD` e `bpf_map_update_elem` – di registrare un nuovo modello a runtime in ciascuna pipeline. Per la pipeline hardcoded, che non ha alcuna via incrementale per costruzione, “aggiungere un modello” significa rigenerare il sorgente C e ricompilare da zero: un’operazione che costa, in media, circa 1.1 secondi. Per le pipeline template e modular, che invece si limitano a scrivere le nuove mappe di pesi, lo stesso aggiornamento costa rispettivamente circa 1.6 e 1.1 millisecondi: da 700 a mille volte più economico. Questo è esattamente il prezzo della flessibilità: template e modular pagano un costo per pacchetto più alto in cambio di un costo di aggiornamento del modello quasi nullo.

4.7 Come sono cambiati i numeri nel tempo

Vale la pena essere trasparenti sul fatto che i numeri della Tabella 4.1 non sono gli stessi misurati la prima volta che le tre pipeline sono state confrontate, all’inizio di questo lavoro, e le differenze raccontano qualcosa di utile. In una delle prime misurazioni, prima di introdurre l’input vector descrittore-driven discusso nel Capitolo 5 e prima di correggere la metodologia di misura discussa nel Capitolo 6, i numeri erano: 963, 5951 e 5931 istruzioni per P1, P2 e P3 rispettivamente; 9.0, 150.0 e 163.0 letture di mappa per pacchetto; 85, 476 e 772 nanosecondi di latenza.

Confrontando questi numeri con quelli finali si vedono due effetti distinti, ed è importante non confonderli. Il primo è un effetto di codice reale: le letture di mappa per pacchetto sono *diminuite* in tutte e tre le pipeline (da 9 a 4 per P1, da 150 a 147 per P2, da 163 a 160 per P3), ed è la firma diretta dell’ottimizzazione che consolida N letture di mappa in una sola, descritta nella Sezione 5.2.3. Il secondo effetto, molto più grande in valore assoluto, riguarda la latenza, e qui la causa non è (solo) un cambiamento di codice ma un cambiamento di *metodologia*: le prime misure si basavano su un singolo campione di `BPF_PROG_TEST_RUN`, che come racconta il Capitolo 6 si è rivelato affetto da un rumore di sistema capace di alterare il risultato anche di un fattore venti. I numeri finali, ottenuti come minimo su più campioni indipendenti, sono sistematicamente più bassi e, soprattutto, stabili tra un’esecuzione e l’altra – cosa che i primi numeri non erano. Il conteggio di istruzioni statiche di P2 e P3 è invece cresciuto in modo sostanziale rispetto alle primissime misure: è il prezzo, discusso anche nelle note di validazione del progetto, di aver reso l’input vector descrittore-driven anche in quelle due pipeline, non solo nella hardcoded – un ciclo generico per-feature, srotolato su un numero massimo di feature dichiarabili, pesa più delle poche righe cablate che sostituisce.

4.8 Il costo della flessibilità

Il design space può essere riassunto in una frase: non esiste un'unica implementazione naturale per portare un modello neurale nel data plane, esiste piuttosto una scelta su quanto specializzare il codice sul singolo modello. La pipeline hardcoded massimizza le prestazioni a scapito della flessibilità: aggiornare il modello significa ricompilare e ricaricare l'intero programma. La pipeline template trova un compromesso pratico: un solo programma per un'intera famiglia di architetture, aggiornabile scrivendo solo una mappa. La pipeline modular massimizza la flessibilità, rendendo dinamiche sia la larghezza sia la profondità del modello, al prezzo del numero di tail call più alto e delle prestazioni per pacchetto più basse.

Proprio perché la pipeline hardcoded vince così nettamente sul piano prestazionale – ed è quella più vicina al caso d'uso in cui il modello è noto in anticipo, come nello scenario di routing che si sta studiando – il resto di questa tesi si concentra su di essa: sui suoi limiti iniziali, su come sono stati superati senza sacrificare le prestazioni, e su un secondo esperimento, altrettanto interessante, sul se convenga renderla più larga o più profonda a parità di risorse.

Capitolo 5

L'evoluzione della pipeline hardcoded

La pipeline hardcoded, come si è visto, è la scelta vincente sul piano prestazionale: aggiunge appena 14 nanosecondi al pavimento imposto dal solo framework XDP, e questo la rende la candidata più naturale per uno scenario in cui i modelli sono noti in anticipo e cambiano relativamente di rado. Proprio perché è la pipeline su cui puntare, ha però senso investire nel superare i suoi due limiti più evidenti, che il relatore ci ha segnalato esplicitamente durante una revisione del lavoro: l'input vector era cablato su un solo scenario di rete, e ogni aggiornamento del modello comportava un costo di ricompilazione molto alto. Questo capitolo racconta come sono stati affrontati entrambi i problemi, e un terzo filone di lavoro, non richiesto ma che è emerso naturalmente dal secondo: capire se, a parità di budget di pesi, convenga rendere il modello più largo o più profondo.

5.1 I due limiti di partenza

Nella versione originale della pipeline hardcoded l'input vector era fisso nella forma 65-x-x-7: sessantacinque feature in ingresso, sempre nello stesso ordine (`link_state`, `iface`, `ttl`, `node`), pensate per lo scenario di routing basato su FRR che si stava studiando. Un modello pensato per uno scenario di rete diverso – con feature diverse, o con un numero diverso di feature dello stesso tipo – semplicemente non poteva essere caricato senza riscrivere a mano il generatore di codice.

Il secondo limite era il costo di ricompilazione: ogni volta che un modello veniva registrato per la prima volta, o che i suoi pesi cambiavano, il generatore produceva un nuovo sorgente C, che veniva compilato da zero da `clang` sul nodo stesso, al momento del caricamento. Nella misura effettuata con `bench_model_add.py` questo costo era dell'ordine del secondo, contro pochi millisecondi per le pipeline template e modular.

Il vincolo che abbiamo tenuto fermo per tutto il lavoro, e che vale la pena esplicitare, è che per i modelli noti a priori – il caso d'uso che stiamo effettivamente studiando – a parità di tipo di feature la dimensione è fissa all'interno della stessa rete: è una

proprietà della *topologia*, condivisa da tutti i nodi della stessa rete, non un parametro che ciascun modello può scegliere liberamente. I modelli scelgono solo *quali* tipi di feature usare, non quanto grande sia ciascun tipo.

5.2 Un input vector generico

La soluzione al primo limite è stata introdurre un catalogo dei tipi di feature che il generatore di codice sa costruire, e un descrittore per-modello che elenca, in ordine, quali tipi quel particolare modello usa. Il catalogo distingue tre categorie (*kind*) di feature: *scalar* (un singolo valore letto dal pacchetto in transito, come il TTL), *dense_vector* (un piccolo vettore di valori letto in un colpo solo da una mappa BPF condivisa, come `link_state` o una nuova feature `queue_occupancy` aggiunta in questa fase) e *onehot* (esattamente un indice attivo su un range di dimensione nota, come `iface` o `node`).

Un modello dichiara il proprio descrittore come una semplice lista ordinata di tipi, ad esempio:

Listing 5.1: Descrittore per-modello: solo i TIPI, non le dimensioni

```
1 features = ["link_state", "queue_occupancy", "ttl"]
2 hidden_dims = [4, 4]
3 n_out = 7
4 # N_IN = somma delle dimensioni dei tipi elencati
```

mentre la *dimensione* di ciascun tipo – quante interfacce ha `link_state`, quanti nodi ha `node` – viene letta da un file separato, condiviso da tutti i nodi della stessa rete, chiamato `topology_config`. Le eventuali chiavi topologiche presenti nel file del singolo modello vengono deliberatamente ignorate: la topologia è una proprietà della rete, non del modello, ed è importante che questa distinzione sia rispettata nel codice, non solo nella documentazione.

Il generatore di codice C, per ciascun tipo di feature nel descrittore, produce un frammento diverso a seconda del *kind*: per una feature scalare, semplicemente legge il valore dal pacchetto; per una feature dense, emette un'unica lettura di mappa seguita da un certo numero di letture dirette dal valore restituito; per una feature one-hot, emette il blocco `switch` descritto nel capitolo precedente. In questo modo, qualunque combinazione di tipi venga dichiarata da un modello, il codice generato costruisce esattamente quell'input vector, feature per feature, mantenendo la stessa filosofia di inferenza completamente srotolata che caratterizza la pipeline hardcoded.

Per non rompere nulla di quanto già funzionava, se un modello non dichiara affatto un descrittore il generatore ricade sul comportamento storico – [`link_state`, `iface`, `ttl`, `node`] – producendo un programma byte-identico a quello di prima: abbiamo verificato questa proprietà confrontando riga per riga l'output del nuovo generatore

con quello del vecchio a parità di pesi, trovando solo differenze cosmetiche nell'ordine dei termini della somma, mai una differenza sostanziale.

5.2.1 Coerenza tra checkpoint e topologia

Un rischio concreto di questo sistema più flessibile è che un modello venga addestrato assumendo una certa dimensione dell'input (ad esempio $N_{IN} = 11$, per un descrittore ridotto) e venga poi caricato su un nodo la cui topologia produce una dimensione diversa (ad esempio $N_{IN} = 13$, perché quella rete ha un numero diverso di code per interfaccia). Prima di introdurre un controllo esplicito, questo tipo di disallineamento passava completamente inosservato: il programma C veniva generato e caricato senza errori, semplicemente calcolando un'inferenza sbagliata, perché i pesi allenati per un layout venivano applicati a un input costruito con un layout diverso.

Per evitare questo scenario abbiamo introdotto un controllo bloccante, eseguito prima ancora di generare una riga di codice C: si legge la dimensione dell'input attesa dal checkpoint del modello (la forma del primo strato lineare, cioè per cui il modello è stato effettivamente addestrato) e la si confronta con la dimensione che la topologia della rete produrrebbe per quel descrittore di feature; se le due cifre non coincidono, viene sollevata un'eccezione esplicita prima di procedere. Un errore netto ed esplicito, in questo caso, è decisamente preferibile a un modello che gira silenziosamente su un input vector incompatibile e sceglie next-hop sbagliati senza che nessuno se ne accorga.

5.2.2 Un esempio concreto

Per rendere più chiaro come tutto questo si traduca in comportamento a runtime, vale la pena seguire un pacchetto destinato a un modello con un descrittore ridotto, ad esempio `[link_state, queue_occupancy, ttl]` (rispettivamente 6, 4 e 1 componenti, per un totale $N_{IN} = 11$). L'header IPA dentro il pacchetto dichiara il `model_id`, e per quel modello un numero di tipi di feature (`n_feat`) pari a 3, ciascuno con il proprio codice e la propria dimensione: il pacchetto *dice* quali feature quel modello usa, il nodo sa *da dove* leggerle e *quanto grande* è ciascuna, in base al proprio `topology_config`.

A runtime, il programma legge le prime 6 componenti di `link_state` da una mappa condivisa, le successive 4 di `queue_occupancy` da un'altra mappa, e l'ultima componente direttamente dal campo TTL del pacchetto in transito, costruendo così un vettore di 11 componenti che viene poi dato in pasto a un MLP $11 \rightarrow 4 \rightarrow 4 \rightarrow 7$, esattamente con la stessa logica di argmax e risoluzione via `mac_table` già descritta per il caso a 65 componenti.

5.2.3 Ottimizzazione dell'hot path: da N letture a una sola

Nella prima versione di questo meccanismo, ogni singolo valore di una feature dense veniva letto con una chiamata separata a `bpf_map_lookup_elem()`: sei chiamate per `link_state`, quattro per `queue_occupancy`, fino a dieci helper call per pacchetto solo per le feature dense. Ci siamo accorti che questo era inutilmente costoso, dato che tutti i valori di una stessa feature vivono comunque in un'unica entry della mappa: basta ridefinire la mappa in modo che il suo valore sia una struttura che contiene l'intero vettore, invece di un singolo intero per slot, e leggerla con un'unica `lookup` su una chiave fissa (`zero`), seguita da un certo numero di letture dirette sul puntatore restituito – letture che, a differenza delle helper call, non hanno alcun costo aggiuntivo per il verifier.

Listing 5.2: Da N lookup a 1: mappa struct-valued

```
1 struct ls_vec { __u32 v[6]; };
2 BPF_ARRAY(link_state, struct ls_vec, 1);
3
4 int z = 0;
5 struct ls_vec *p = link_state.lookup(&z);
6 if (p) { ls0 = p->v[0]; ls1 = p->v[1]; /* ... */ }
```

Questa modifica è stata applicata a tutte e tre le pipeline, non solo alla hardcoded, senza toccare in nessun modo l'architettura a dispatcher più tail call: cambia solo *come* viene letta la mappa. Per la pipeline hardcoded il risultato è una riduzione di cinque letture di mappa per pacchetto, verificata mantenendo intatta l'inferenza (stessa classe scelta, confrontata con il riferimento Python) su tutta la suite di test.

5.3 AOT-literal: separare la compilazione dal deploy

Il secondo limite – il costo di ricompilazione – è più sottile da risolvere, perché è intrinseco alla natura stessa della pipeline hardcoded: se i pesi sono letterali nel codice C, un cambio di pesi richiede necessariamente una nuova compilazione. Non si tratta quindi di eliminare la compilazione, ma di capire se sia possibile spostarla *fuori dal percorso critico*.

La pipeline hardcoded, nella sua forma originale, usa BCC (BPF Compiler Collection): una libreria che include al proprio interno LLVM/clang e compila il sorgente C a runtime, ogni volta che il programma viene caricato, direttamente sul nodo che farà da datapath. Il costo misurato di questa compilazione a runtime, isolando il solo tempo di `clang`, è di circa 1660 millisecondi per modello: pochissimo se il modello cambia una volta ogni tanto, ma un costo enorme se lo si moltiplica per molti nodi che

devono ricevere lo stesso aggiornamento, o se si vuole un vero hot-swap del modello sul datapath vivo.

L'osservazione chiave è che il numero di compilazioni non deve necessariamente cambiare – resta una per modello, dato che i pesi restano letterali – ma *dove* e *quando* questa compilazione avviene sì. Abbiamo quindi costruito un percorso alternativo, che chiamiamo AOT-literal (Ahead-Of-Time), che genera lo stesso tipo di sorgente C con i pesi come letterali, ma nel dialetto `libbpf` invece che nel dialetto BCC, e lo compila con `clang offline`, su una build box separata dal nodo che farà da datapath, producendo un file oggetto `.o` già pronto. Il nodo datapath, a quel punto, deve solo aprire e caricare questo file già compilato – un'operazione ordini di grandezza più leggera del compilare da zero.

I numeri misurati confermano questa intuizione: la build offline costa, a seconda della run, tra i 90 e i 555 millisecondi di `clang`, ma questo tempo è pagato una sola volta sulla build box, fuori dal percorso critico. Sul nodo datapath, aprire il file oggetto e caricarlo nel kernel (verifica più JIT) costa, nell'ultima misura effettuata, circa 6 millisecondi in totale – contro i 1660 millisecondi della compilazione BCC a runtime. Le prestazioni per pacchetto restano sostanzialmente identiche a quelle della pipeline hardcoded via BCC, perché l'architettura generata (dispatcher più tail call più doppio parsing) è la stessa e i pesi restano letterali, con lo stesso beneficio di strength reduction che il compilatore applica automaticamente su costanti note (una moltiplicazione per zero sparisce del tutto, una moltiplicazione per una potenza di due diventa uno shift): l'ultima misura, fatta con la stessa metodologia usata per il resto della tabella comparativa, dà 1010 istruzioni totali (28 di dispatcher più 982 del modello), 42 nanosecondi per pacchetto e 23.8 milioni di pacchetti al secondo.

Su richiesta esplicita del relatore, questo percorso AOT-literal non è rimasto una semplice alternativa selezionabile a piacere, ma è diventato l'*unico* meccanismo con cui la pipeline hardcoded viene distribuita e attaccata in produzione, in sostituzione completa del vecchio percorso basato su BCC: se ne parla in dettaglio, insieme alle difficoltà tecniche che questa sostituzione ha comportato sul laboratorio Kathara, nella Sezione 5.7 più avanti in questo stesso capitolo.

Nella sua prima versione, inoltre, il generatore AOT-literal copriva solo il descrittore di feature storico (il classico 65-4-4-7): il lavoro descritto poco sopra sul catalogo di feature era stato implementato solo nel dialetto BCC. Una parte del lavoro è consistita proprio nel portare i tre generatori di codice per *kind* di feature (scalare, dense, one-hot) anche nel dialetto `libbpf`, cosicché qualunque descrittore accettato dal percorso BCC sia ora compilabile anche in AOT – la differenza tra i due dialetti è puramente sintattica (funzioni “nude” come `m.lookup(&k)` in BCC contro `struct {...} SEC(".maps")` e `bpf_map_lookup_elem(&m, &k)` in `libbpf`), non concettuale.

5.4 Profondità variabile

Un limite meno visibile, ma altrettanto rigido, era che sia il generatore di codice BCC sia quello AOT assumevano sempre esattamente due hidden layer: le variabili nel codice si chiamavano letteralmente `n_h1` e `n_h2`, e tutta la logica di generazione – lo slicing dei pesi, l'emissione del primo layer, del secondo, dello strato di output – era scritta esplicitamente per due strati, non per un numero arbitrario.

Abbiamo generalizzato entrambi i generatori (e, in modo speculare, anche il riferimento Python usato dalla suite di test per verificare la correttezza dell'inferenza) in modo che `hidden_dims` sia una sequenza di lunghezza qualsiasi: due valori come prima, ma anche uno solo, quattro, o addirittura nessuno (un modello puramente lineare, senza hidden layer). Il layout dei pesi diventa generico per-layer – per ogni coppia di strati consecutivi, un blocco di $n_{prev} \times n_{cur}$ pesi seguito da n_{cur} bias, uno strato dopo l'altro – e questo layout generico si riduce esattamente al vecchio schema quando `hidden_dims` vale (4,4). Abbiamo verificato questa proprietà in due modi: confrontando byte per byte il codice C generato per il caso a due strati con quello prodotto dal generatore precedente alla modifica (identico, a meno di righe vuote), e confrontando numericamente l'output del nuovo riferimento Python generalizzato con la vecchia implementazione esplicita a due strati, su pesi e input casuali, ottenendo lo stesso identico risultato.

Questa generalizzazione, di per sé un dettaglio tecnico, è quella che ha reso possibile l'esperimento descritto nella sezione seguente, che è probabilmente il contributo più originale di questa parte del lavoro.

5.5 Larghezza contro profondità: uno studio sperimentale

Una volta reso possibile generare modelli con un numero arbitrario di hidden layer, è emersa naturalmente una domanda: a parità di numero di pesi (e quindi, grosso modo, a parità di capacità del modello), conviene concentrare quei pesi in pochi strati larghi, o distribuirli su molti strati stretti? È una domanda con un risvolto molto pratico per la pipeline hardcoded, dato che l'intera inferenza vive dentro un'unica funzione C srotolata, soggetta al limite di 512 byte di stack che il verifier eBPF impone su ogni singola funzione.

5.5.1 Metodologia

Per rispondere a questa domanda abbiamo costruito uno script di benchmark dedicato che confronta, a tre livelli di budget di pesi crescente (circa 300, 1200 e 4700 pesi),

una forma “larga” (un solo hidden layer) contro due forme “profonde” (quattro e otto hidden layer), avendo cura di dichiarare esplicitamente lo scarto reale tra i budget di pesi delle diverse forme, mai di darlo per scontato come uguale. Per evitare che la conclusione dipendesse in modo nascosto dal particolare descrittore di feature usato di default (che contiene una feature one-hot piuttosto costosa, `node`, con 52 possibili valori), abbiamo ripetuto lo stesso esperimento su quattro descrittori diversi: quello storico con due feature one-hot, uno senza nessuna feature one-hot, uno con una sola one-hot piccola e uno con una sola one-hot grande.

La parte più istruttiva di questo lavoro, però, non è stata l'esperimento in sé ma il processo per arrivare a una misura affidabile, raccontato nella sezione successiva.

5.5.2 Il problema del rumore di misura

Il primo tentativo di misurare la latenza per ciascuna configurazione, con un singolo campione di `BPF_PROG_TEST_RUN`, ha prodotto numeri estremamente instabili: la stessa identica configurazione, misurata due volte a distanza di pochi minuti, poteva restituire valori che differivano anche di un fattore venti, senza nessuna correlazione apparente con il numero di istruzioni della configurazione. Questo fenomeno non era limitato al singolo script di benchmark: lo stesso problema è emerso, in modo ancora più visibile perché il relatore l'ha notato direttamente, nella suite di test principale del progetto, dove il throughput misurato per la pipeline hardcoded oscillava, da un'esecuzione all'altra, tra circa 10 e 25 milioni di pacchetti al secondo, e quello della configurazione più semplice (solo parsing e redirect) tra 10 e 50.

La spiegazione di questo comportamento è che il rumore introdotto da un ambiente condiviso – interruzioni dello scheduler, altri processi sulla stessa macchina, variazioni di frequenza della CPU – è *a senso unico*: può solo rallentare un singolo campione di misura, mai accelerarlo al di sotto del suo vero costo. Questo rende il *minimo* su più campioni indipendenti la statistica corretta da riportare, non la media né la mediana: è lo stesso principio su cui si basano strumenti di benchmarking consolidati come `hyperfine` o Google Benchmark. Abbiamo quindi corretto sia lo script dedicato al confronto larghezza/profondità sia, cosa più importante, la suite di test principale del progetto, facendo eseguire ciascuna misura più volte (rispettivamente quindici e sette ripetizioni indipendenti) e riportando il minimo, insieme al cinquantesimo e novantesimo percentile per trasparenza, invece di un singolo numero che poteva essere fuorviante. Dopo questa correzione i numeri di throughput della suite principale, riportati nella Tabella 4.1 del capitolo precedente, sono risultati stabili tra esecuzioni successive.

Un secondo problema, distinto dal primo, riguardava le configurazioni più estreme dell'esperimento (i budget di pesi più grandi): alcune di queste sfiorano il limite di 512 byte di stack per funzione imposto dal verifier, e quando questo accade il bac-

kend LLVM usato da BCC termina con un errore fatale a livello di processo, non con un'eccezione Python catturabile – un singolo tentativo sbagliato di stimare in anticipo quali configurazioni avrebbero sfiorato lo stack (un primo tentativo con una formula che considerava solo il primo hidden layer, un secondo che sommava tutti gli strati) si è rivelato ripetutamente smentito dal comportamento reale del compilatore, che a volte riesce a riutilizzare lo spazio di stack tra variabili con vita non sovrapposta e a volte no, in modo non facilmente prevedibile a priori. La soluzione più robusta è stata rinunciare a prevedere l'esito ed eseguire ciascuna configurazione in un sottoprocesso isolato: se il sottoprocesso muore per un errore fatale del compilatore, viene semplicemente segnato come fallito e l'intero esperimento prosegue con le configurazioni successive, invece di interrompersi.

5.5.3 Risultati

La Tabella 5.1 riassume i risultati, espressi come minimo su più trial indipendenti, per il descrittore di feature storico; i tre descrittori aggiuntivi hanno mostrato lo stesso ordinamento qualitativo, il che rafforza la fiducia che la conclusione non dipenda dal particolare mix di feature usato.

Tabella 5.1: Larghezza contro profondità a parità di budget di pesi (minimo su 15 trial, descrittore storico)

Configurazione	Pesi	Latenza minima (ns/pkt)	Note
Baseline / largo, 1×4	$\sim 300\text{--}320$	44–56	riferimento
Profondo, 8×3	~ 310	57–76	overhead contenuto
Largo, 1×16	~ 1175	103–111	sempre il più veloce
Profondo, 4×11	~ 1206	203–236	circa $2 \times$ più lento
Profondo, 8×9	~ 1294	269–301	$2.5\text{--}3 \times$ più lento
Qualunque forma	$\sim 4700\text{--}4780$	—	sempre fallito

Il risultato è netto e coerente su tutti e quattro i descrittori testati: a parità di budget di pesi, allargare un modello (concentrare i pesi in un numero minore di strati più larghi) è sempre più veloce che approfondirlo, e la differenza cresce con il budget, arrivando a un fattore due o tre al livello intermedio di budget testato. Ogni strato nascosto aggiuntivo sembra introdurre un costo fisso, dell'ordine di 15-40 nanosecondi, dovuto alla transizione tra uno strato e il successivo e all'attivazione ReLU, un costo che è risultato indipendente dalla particolare composizione di feature del descrittore.

Il secondo risultato, forse ancora più interessante dal punto di vista architetturale, è che al livello di budget più alto testato *ogni* configurazione – larga o profonda che fosse – ha sfiorato il limite di stack ed è fallita in compilazione, indipendentemente dal descrittore. Questo indica che esiste un tetto di capacità assoluto per l'architettura har-

dcoded così come è oggi costruita (tutta l'inferenza srotolata dentro un'unica funzione, pesi come letterali): non è una questione di scegliere bene tra larghezza e profondità, è un limite strutturale del design, superabile solo spostando gli array più grandi in una mappa `PERCPU_ARRAY` – suggerimento che, non a caso, compare testualmente nel messaggio di errore del compilatore stesso – invece che continuare a redistribuire diversamente lo stesso numero di pesi.

5.6 Isolare il costo puro della tail call

Il confronto tra baseline e pipeline hardcoded discusso nel capitolo precedente (21 contro 35 nanosecondi) attribuisce i 14 nanosecondi di differenza, tutti insieme, alla tail call verso `model_<id>`, al secondo parsing del pacchetto che questa comporta e all'intera rete neurale. Ci siamo chiesti se fosse possibile isolare, tra questi tre contributi, il costo della sola tail call, che è un dettaglio architetturale comune anche alle altre due pipeline e non qualcosa di specifico dell'inferenza.

Per rispondere abbiamo costruito due varianti minime del programma baseline già usato come pavimento di riferimento: la prima esegue esattamente lo stesso parsing e la stessa risoluzione dell'azione di forwarding vista nel capitolo precedente, tutto in un unico programma; la seconda esegue lo stesso identico parsing e la stessa identica azione di forwarding, ma la seconda metà del lavoro (la sola risoluzione dell'azione) è spostata in un programma separato, raggiunto tramite un'unica tail call. Le due varianti sono identiche in tutto tranne che nella presenza o assenza di quell'unico salto, il che rende la differenza tra le due misure il costo isolato della tail call in sé, scorporato da qualunque aritmetica di inferenza o da un secondo parsing del pacchetto. Con la stessa metodologia a più campioni indipendenti descritta nel Capitolo 6, questo confronto ha permesso di scomporre più precisamente da dove venga il costo osservato tra baseline e pipeline hardcoded, distinguendo il contributo strutturale del meccanismo di tail call – comune a tutte le pipeline con più di un programma – da quello, ben più consistente, dell'aritmetica del modello e del secondo parsing.

5.7 AOT-literal in sostituzione di BCC per il deploy

Una volta che il percorso AOT-literal è arrivato a coprire qualunque descrittore di feature e qualunque profondità, esattamente come faceva il percorso basato su BCC, il relatore ha chiesto esplicitamente che AOT-literal *sostituisse* BCC come meccanismo di deploy della pipeline hardcoded, e non restasse semplicemente un'alternativa opzionale selezionabile a riga di comando. In una prima versione di questo lavoro avevamo scelto di far convivere i due percorsi, ragionando sui rischi pratici dell'ambiente di laboratorio; una volta chiarito che la richiesta riguardava specificamente il percorso di

deploy live – cioè cosa gira davvero, in produzione, sul nodo che fa da datapath – e non l'infrastruttura di test interna, abbiamo effettivamente rimosso BCC da quel percorso.

Concretamente, questo ha richiesto di risolvere il vincolo che ci aveva inizialmente convinti a tenere entrambi i percorsi: i nodi del laboratorio Kathara non hanno né `clang` né `libbpf` installati, mentre BCC incorpora al proprio interno LLVM/clang come libreria, senza bisogno di questi strumenti installati separatamente. Il percorso AOT-literal, per costruzione, non ha mai avuto bisogno di eseguire `clang` sul nodo datapath – la generazione e la compilazione del sorgente C avvengono offline, su una build box separata, e sul nodo arriva solo il file oggetto già compilato – ma il piccolo loader in linguaggio C che apre quel file oggetto e lo carica nel kernel (`loader_aot.c`) era collegato dinamicamente contro `libbpf`, il che significa che, anche portando sul nodo un binario già compilato altrove, quel binario non sarebbe comunque partito per l'assenza della libreria condivisa `libbpf.so` sul nodo stesso.

La correzione è stata collegare staticamente il loader contro `libbpf` (e le sue stesse dipendenze, `libelf` e `zlib`), lasciando dinamico solo il collegamento alla libreria C standard, che è ragionevole assumere presente su qualunque sistema Linux, a differenza di `libbpf`. Il risultato è un singolo eseguibile autosufficiente: può essere compilato una volta su una build box che ha tutti gli strumenti necessari, e poi copiato ed eseguito su un numero qualunque di nodi datapath che non hanno né `clang` né `libbpf` né alcuna libreria di sviluppo eBPF installata, esattamente come i nodi del laboratorio usato per questo lavoro. Con questa correzione, il percorso di deploy della pipeline hardcoded è oggi interamente basato su AOT-literal: lo script che un tempo permetteva di scegliere tra i due backend a riga di comando (`-hardcoded-backend {aot,bcc}`) è stato semplificato per offrire solo il percorso AOT, e il vecchio loader basato su BCC per il deploy live non viene più invocato in produzione.

Resta un'unica eccezione, volutamente conservata: l'infrastruttura di test descritta nel Capitolo 6 continua a usare BCC internamente per compilare e verificare la correttezza dell'inferenza. Si tratta di una scelta consapevole e distinta dalla questione del deploy: quella suite non gira mai sul nodo datapath in produzione, ma su una macchina di sviluppo o di validazione dove BCC è comunque disponibile, e usarlo lì per compilare al volo il codice da verificare non ha nulla a che vedere con il modo in cui il modello viene effettivamente distribuito e eseguito sulla rete.

5.8 Robustezza del control plane: la risoluzione dei MAC per classe

Un ultimo filone di lavoro, più vicino al control plane che al datapath, riguarda come viene popolata la mappa `mac_table` che traduce la classe scelta dall'argmax in un'azio-

ne di forwarding concreta. In una prima versione, tutte le classi venivano fatte puntare alla stessa interfaccia risolta una sola volta, il che rendeva di fatto ininfluyente quale classe l'inferenza scegliesse: il pacchetto usciva comunque sempre dalla stessa porta. Abbiamo corretto questo comportamento facendo sì che ciascuna classe venga associata, tramite la tabella ARP del sistema, al MAC del vicino effettivamente raggiungibile su *quella specifica* interfaccia di uscita.

Durante una revisione di questo meccanismo abbiamo trovato e corretto due difetti più sottili. Il primo era una doppia lettura della tabella ARP non allineata: la funzione che installa le regole risolveva il MAC del vicino una prima volta, mentre un secondo pezzo di codice, chiamato subito dopo, rileggeva la stessa tabella una seconda volta per decidere quali classi avviare in un polling periodico di aggiornamento – tra le due letture l'ARP poteva nel frattempo risolversi, lasciando una classe non sorvegliata pur essendo ancora sul MAC di riserva, o viceversa. Il secondo difetto era che il thread di aggiornamento periodico smetteva di sorvegliare una classe non appena questa aveva risolto correttamente una prima volta, mentre una entry ARP può scadere, o il vicino dietro una porta può cambiare a seguito di un guasto o di un riavvio, senza che nessuno se ne accorgesse più. Entrambi i problemi sono stati corretti facendo sì che la risoluzione e la decisione su quali classi sorvegliare avvengano nella stessa, unica lettura, e facendo girare il thread di aggiornamento per l'intera vita della pipeline, scrivendo nella mappa solo quando il MAC risolto cambia effettivamente rispetto a quello già scritto.

Capitolo 6

Metodologia di validazione e onestà delle misure

Una parte non piccola di questo lavoro è stata dedicata non a scrivere nuovo codice per il datapath, ma a chiedersi se le misure raccolte fossero davvero affidabili, e a costruire test che lo fossero di più. Questo capitolo raccoglie quel lavoro.

6.1 BPF_PROG_TEST_RUN come banco di prova indipendente dalla rete

Il laboratorio sperimentale usato per questo progetto è basato su Kathara, uno strumento che emula una topologia di rete facendo girare ciascun nodo come un container Docker leggero, condividendo però il kernel del sistema host – ed è proprio questo che permette di usare eBPF/XDP dentro i container. La topologia usata, importata da SNDlib, è *Germany50*: cinquanta nodi di dorsale più due host, con grado massimo di un nodo pari a sei.

Durante il lavoro ci siamo scontrati con un limite pratico di questo fabric: il traffico UDP sulla porta 9999 usato dal protocollo IPA non viene consegnato end-to-end in ogni condizione – in particolare, indirizzato all’indirizzo di loopback di un nodo lontano più di un salto, il traffico semplicemente non arriva, mentre indirizzato all’indirizzo IP dell’interfaccia direttamente connessa a un vicino sì. Per questo motivo la verifica sistematica di correttezza dell’inferenza si appoggia non su traffico di rete reale, ma su `BPF_PROG_TEST_RUN`: una primitiva che permette di eseguire un programma XDP già caricato nel kernel su un pacchetto sintetico, direttamente dal kernel stesso, ottenendo sia il verdetto del programma sia una misura di durata, senza dover attraversare nessuna scheda di rete reale.

Il criterio di correttezza usato in tutta la suite è semplice da enunciare ma robusto: si pre-installa la mappa di forwarding con le azioni attese, si esegue il programma una

volta e si controlla che il verdetto restituito sia coerente con un redirect o un drop, e che il contatore per-classe della classe scelta si sia effettivamente incrementato – confrontando sempre la classe scelta dal kernel con quella calcolata indipendentemente da un riferimento in Python che replica esattamente la stessa aritmetica intera.

6.2 Perché una sola architettura non basta

La suite di test principale del progetto, per lungo tempo, verificava correttezza e prestazioni usando esclusivamente il modello di riferimento a forma fissa 65-4-4-7. Questo lasciava scoperto proprio il tipo di affermazione più forte fatta nel Capitolo 4 e nel Capitolo 5, cioè che le pipeline gestiscano correttamente architetture di forma arbitraria, non solo quella di riferimento.

Abbiamo quindi esteso la suite affinché verifichi anche architetture alternative: per la pipeline hardcoded, due programmi compilati separatamente con profondità diverse (uno con un solo hidden layer, uno con tre), controllati contro il riferimento Python generalizzato descritto nel Capitolo 5; per le pipeline template e modular, un modello sintetico con forma diversa da quella di riferimento, registrato *insieme* al modello reale nello stesso oggetto compilato, per dimostrare che la registrazione concorrente di più architetture funzioni davvero e non solo in teoria.

6.3 Cosa dice la letteratura sulla valutazione di un datapath eBPF/XDP

Prima di consolidare la metodologia di misura abbiamo cercato riscontro in letteratura su come si valutano correttamente sistemi di questo tipo. Il paper originale che ha introdotto XDP [1] definisce le metriche di riferimento standard per questo genere di sistemi: throughput massimo in milioni di pacchetti al secondo a CPU satura, latenza per-pacchetto aggiunta dall’hook rispetto al percorso nudo del kernel, e overhead computazionale del solo codice eBPF, isolato dal costo di I/O.

Un secondo riferimento, più specificamente rivolto ai limiti metodologici piuttosto che alle metriche in sé, è la raccolta di “crimini del benchmarking” sistematizzata da Heiser [2], che cataloga in modo sistematico gli errori più comuni che invalidano un confronto sperimentale in sistemi come questo: dalla scalatura della frequenza della CPU non disabilitata, ai C-state del processore che introducono latenza di risveglio non contabilizzata, fino al contabilizzare il tempo per tick, che può sottostimare drasticamente il costo reale di un softirq – un problema descritto concretamente, per il caso specifico di XDP, in un intervento a USENIX LISA 2021 [3], dove viene mostrato un

caso in cui l'utilizzo di CPU riportato dagli strumenti standard di sistema sottostimava il costo reale di un fattore sessanta.

Questi riferimenti sono serviti soprattutto a inquadrare correttamente i limiti del nostro stesso ambiente sperimentale: il laboratorio Kathara gira su una macchina virtuale, senza alcun controllo sulla frequenza della CPU, sui C-state o sull'affinità delle interruzioni. I numeri assoluti riportati in questa tesi – nanosecondi per pacchetto, milioni di pacchetti al secondo – non sono quindi direttamente comparabili con quelli di un paper su hardware dedicato e isolato. Quello che resta pienamente difendibile con questo tipo di ambiente, ed è la misura su cui si basano tutte le conclusioni di questa tesi, è il confronto *relativo* tra le pipeline, misurate sullo stesso nodo, nelle stesse condizioni.

Vale la pena segnalare, infine, che il problema di eseguire inferenza neurale dentro il kernel non è isolato a questo progetto: esistono lavori recenti e indipendenti che esplorano la stessa direzione, ad esempio inferenza di reti convoluzionali leggere direttamente in eBPF per la classificazione del traffico su dispositivi edge [4], o classificatori basati su reti neurali quantizzate e alberi decisionali eseguiti sia in kernel sia in user space per il packet processing [5]: un segnale che la direzione di questo lavoro non è isolata, ma si inserisce in un filone di ricerca attivo.

6.4 Alcuni tentativi abbandonati

Non tutto quello che abbiamo provato ha funzionato, ed è onesto dirlo. Abbiamo tentato di costruire un test end-to-end che generasse traffico reale (non sintetico via `BPF_PROG_TEST_RUN`) e ne misurasse il throughput realmente consegnato a una pipeline attaccata dal vivo, e un secondo test che simulasse un guasto reale su un'interfaccia – non una scrittura diretta nella mappa `link_state` – verificando che il meccanismo di fast-reroute descritto nel Capitolo 4 reagisse davvero a un evento di rete reale. Entrambi i tentativi si sono scontrati con limiti concreti dell'ambiente di laboratorio: il primo ha rischiato più volte di produrre numeri completamente fuorvianti, perché un socket UDP può riportare un throughput di invio molto alto anche quando, per un errore di configurazione dell'indirizzo di destinazione, nessun pacchetto raggiunge davvero il nodo ricevente; il secondo si appoggiava a uno strumento di introspezione delle mappe BPF (`bpftool`) che si è rivelato assente dalle immagini dei container usate nel laboratorio. Abbiamo deciso di non insistere oltre su questi due test e di rimuoverli, preferendo lasciare nella tesi una descrizione onesta del perché non hanno funzionato piuttosto che un risultato non completamente verificabile.

Capitolo 7

Conclusioni e sviluppi futuri

Questo lavoro ha esplorato un design space a tre pipeline per portare l'inferenza di un modello neurale compatto dentro il datapath eBPF/XDP, confermando sperimentalmente il trade-off atteso tra prestazioni di picco e flessibilità di aggiornamento a runtime: la pipeline hardcoded aggiunge appena 14 nanosecondi al pavimento imposto dal solo framework XDP, mentre le pipeline template e modular pagano un costo per pacchetto crescente in cambio di un aggiornamento del modello centinaia di volte più economico.

Concentrandosi poi sulla pipeline vincente sul piano prestazionale, il lavoro ha mostrato che i suoi due limiti principali – un input vector cablato su un solo scenario e un costo di ricompilazione molto alto – non sono intrinseci alla scelta di specializzare il codice sul singolo modello, ma conseguenze di scelte implementative specifiche, entrambe superabili senza rinunciare alle prestazioni: un catalogo di feature con un descrittore per-modello rende l'input vector flessibile a costo zero rispetto al caso storico, e separare la compilazione (che resta necessaria, ma può avvenire offline) dal deploy riduce il costo di aggiornamento sul nodo datapath da circa 1660 millisecondi a pochi millisecondi, a parità di prestazioni per pacchetto.

Lo studio sulla profondità variabile ha aggiunto un risultato non scontato: a parità di numero di pesi, allargare un modello è sistematicamente più veloce che approfondirlo, con un costo fisso per strato aggiuntivo indipendente dal descrittore di feature usato, e con un tetto di capacità assoluto – non aggirabile scegliendo diversamente tra larghezza e profondità – imposto dal limite di stack per funzione del verifier eBPF.

Non meno importante, gran parte del lavoro descritto nell'ultimo capitolo è consistito nel mettere in discussione l'affidabilità delle misure stesse: un rumore di sistema a senso unico può far apparire una pipeline fino a venti volte più lenta di quanto sia realmente, se non si usano più campioni indipendenti e la statistica giusta (il minimo, non la media) per filtrarlo. È una lezione che vale probabilmente più della singola cifra di throughput misurata, ed è per questo che le è stato dedicato un intero capitolo.

Come sviluppi futuri, il limite di stack che impedisce di scalare la pipeline hardcoded

oltre un certo budget di pesi resta un problema aperto e concreto: spostare gli array di attivazione più grandi in una mappa `PERCPU_ARRAY`, come suggerisce lo stesso messaggio di errore del verifier, sembra la via più naturale e non è stato ancora esplorato in questo lavoro. Resta inoltre aperta la possibilità di completare, con un ambiente di laboratorio meno limitato dell'attuale (con strumenti di introspezione delle mappe disponibili e un fabric che consegna traffico reale end-to-end), i due test di validazione descritti come tentativi abbandonati nel capitolo precedente, che avrebbero un valore genuino per confermare su traffico reale, e non solo su `BPF_PROG_TEST_RUN`, sia il throughput relativo tra le pipeline sia il comportamento di fast-reroute su un guasto vero.

Bibliografia

- [1] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, D. Miller. *The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel*. Proceedings of the 14th International Conference on emerging Networking EXperiments and Technologies (CoNEXT), 2018.
- [2] G. Heiser et al. *Benchmarking Crimes: An Emerging Threat in Systems Security*. arXiv:1801.02381, 2018.
- [3] Z. Jones. *Performance Analysis of XDP Programs*. USENIX LISA21, 2021.
- [4] *In-Kernel CNN Inference for Edge Devices: An eBPF-Based Approach to Low-Latency and Resource-Efficient Processing*. Springer, 2025.
- [5] *Practicality of in-kernel/user-space packet processing empowered by lightweight neural network and decision tree*. Computer Networks, 2024.